Reference and Practice Handout

Lecture-02: Data Movement

Computer Organization & Assembly Language Programming
CS 2160 || Fall 2026
Instructor: Himon Thakur



File Transfer

(Randall Munroe; CC BY-NC 2.5; https://xkcd.com/949/)

This document is a supplementary resource alongside course materials and class lectures. You are expected to go through and utilize it by yourself. You may read a section, then try solving the short exercises before moving on. Answers are provided at the end.

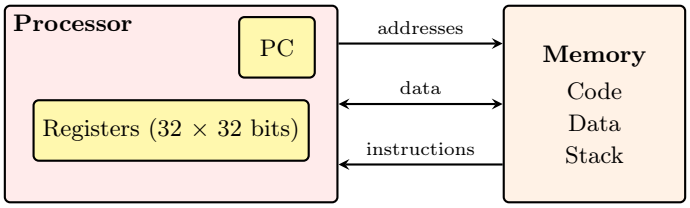
This handout will cover:

- How the processor, its **registers**, and **memory** work together.
- **Loading** data from memory into a register (`lw`, `lh`, `lb`, ...).
- **Addressing modes**: how an instruction says *where* in memory to look.
- **Storing** data from a register back to memory (`sw`, `sh`, `sb`).
- **Endianness**: the order bytes are placed in memory.
- Copying between registers with *pseudo-instructions* (`mv`, `li`).

I used Claude Opus 5.5 to help me format the LaTeX code. All contents were produced, edited, or verified by me.

1. Processor and Memory

The processor can only compute on values that are in its **registers**. Registers are fast but few (32 of them, 4 bytes each). Everything else (the program's code, its data, and its stack) lives in **memory**. Much of a program's work is simply *moving data* between the two.



Every cycle, the processor repeats three steps:

1. **Fetch:** read the next instruction from the memory address in the **program counter (PC)**, a special-purpose register.
2. **Decode:** figure out which operation it is and which registers it uses.
3. **Execute:** do it. An instruction may compute (arithmetic/logic), read or write memory, or talk to an I/O device.

Kind of data movement	Done by	Examples
Instructions: memory → processor	instruction fetch	(automatic, using the PC)
Data: memory → register	**load** instructions	`lw`, `lh`, `lb`
Data: register → memory	**store** instructions	`sw`, `sh`, `sb`
Data: register → register	ALU instructions	`add`, `addi` (`mv`)

1.1. Register cheat sheet

Register	Name	Used for	Kept across calls?
`x0`	`zero`	always 0 (writes are ignored)	–
`x1`	`ra`	return address	no
`x2`	`sp`	stack pointer	yes
`x3`, `x4`	`gp`, `tp`	global / thread pointer	–
`x5-x7`	`t0-t2`	temporaries	no
`x8-x9`	`s0-s1`	saved registers	yes
`x10-x17`	`a0-a7`	arguments; `a0`, `a1` also return values	no
`x18-x27`	`s2-s11`	saved registers	yes
`x28-x31`	`t3-t6`	temporaries	no

There are 7 temporaries in total: `t0`–`t6`.

2. Loading: Memory → Register

A **load** copies a value from a memory address into a register. In C, that is reading through a pointer: `x = *p;`

C	Idea	RISC-V
`x = *p;`	load `x` from address `p`	`lw t1, 0(a0)`

To translate, we first need to know *where* everything is. Assume the pointer `p` (type `int *`) is in `a0`, and `x` goes in `t1`.

```
lw  t1 , 0 ( a0 )
```

- `lw`: "load word". Read 4 bytes (one **word**) from memory.
- `t1`: the **destination** register that receives the value (comes first).
- `0 ( a0 )`: the **address**: the value in `a0` plus an offset of 0.

2.1. Addressing modes: how an instruction names an address

`0(a0)` means "address = `a0` +0". The general form `offset(base)` means:

$$\text{address} = (\text{value in base register}) + \text{offset (a constant)}$$

RISC-V uses two main addressing modes:

Mode	Address is...	Used by
Displacement (base + offset)	a register + a constant offset	loads and stores
PC-relative	the PC + a constant offset	branches, `jal`, `auipc`

The offset is a 12-bit signed constant, so it can be anywhere from −2048 to +2047 bytes. Other ISAs offer more modes, but these two are enough for RISC-V.

Exercise 1

The pointer `p` is in `a1` and `x` should go in `t2`. Write the RISC-V for `x = *p;`.

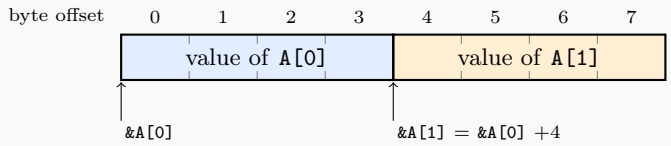
3. Arrays

An array is a row of same-sized elements at **consecutive addresses**. Memory is addressed in **bytes**, so element i is at

$$\texttt{\&A[i]} = \texttt{\&A[0]} + i \times (\text{size of one element in bytes})$$

In C, the array name itself is the address of its first element: `A == &A[0]`.

Example 1: `int A[2]`: each `int` is 4 bytes



With `&A[0]` in `a0` and `x` in `t1`: `x = A[1];` → `lw t1, 4(a0)`

Watch out

The offset counts **bytes**, not elements. `A[1]` of an `int` array is at offset **4**, not 1. For element i, the offset is $4i$ (for a constant i) or must be computed with a shift, `slli t0, t0, 2`, when i is in a register.

3.1. Different sizes: byte, half-word, word

A register is always 32 bits (bit 0 is the least significant bit, bit 31 the most significant). Smaller loads fill only the low bits, then **extend** the value to fill the rest:

- **signed** loads (`lb`, `lh`) copy the sign bit into the upper bits (sign extension);
- **unsigned** loads (`lbu`, `lhu`) fill the upper bits with 0s (zero extension).

Load	Meaning	Bytes	Fills upper bits with	Typical C type (RV32)
`lb`	load byte	1	copies of the sign bit	`signed char`, `int8_t`
`lbu`	load byte unsigned	1	0s	`char`, `unsigned char`, `uint8_t`
`lh`	load half-word	2	copies of the sign bit	`short`
`lhu`	load half-word unsigned	2	0s	`unsigned short`
`lw`	load word	4	(nothing to fill)	`int`, `long`, pointers

- On RISC-V, a plain `char` is *unsigned* by default, so compilers use `lbu` for it.

- `ld` (load doubleword, 8 bytes) exists only on 64-bit RISC-V (RV64). On RV32, a 64-bit `long long` takes two `lw`s.
- `li` ("load immediate") is not a memory load at all; see Section 6.

Example 2: Same address, different loads

Memory at address `0x200` (in `a0`) holds the bytes `F0 FF 34 12` (lowest address first).

Instruction	Bytes read	Register result	Decimal
`lb t0, 0(a0)`	`F0`	`0xFFFFFFF0`	−16
`lbu t0, 0(a0)`	`F0`	`0x000000F0`	240
`lh t0, 0(a0)`	`F0 FF`	`0xFFFFFFF0`	−16
`lhu t0, 0(a0)`	`F0 FF`	`0x0000FFF0`	65,520
`lh t0, 2(a0)`	`34 12`	`0x00001234`	4,660
`lw t0, 0(a0)`	`F0 FF 34 12`	`0x1234FFF0`	305,463,280

Multi-byte values read "backwards" (`F0 FF` becomes `0xFFF0`) because RISC-V is little-endian. Section 5 explains why.

Exercise 2

`int A[10]` starts at address `0x1000`. What is the address of `A[5]`? Now `short B[10]` starts at `0x2000`; what is the address of `B[5]`, and which instruction loads it into `t0` if `&B[0]` is in `a0`?

Exercise 3

With `&A[0]` in `a0` (`int A[]`), write RISC-V for `x = A[3] + A[4];` with `x` in `t2`.

4. Storing: Register → Memory

A **store** copies a register's value to a memory address. In C, that is writing through a pointer: `*p = x;`

C	Idea	RISC-V
`*p = x;`	store `x` to address `p`	`sw t1, 0(a0)`
`A[1] = x;`	store `x` to address `A` +4	`sw t1, 4(a0)`

Watch out

The register order looks the same as a load, but the data flows the other way.
`lw t1, 0(a0)`: memory → `t1` (`t1` is the *output*).
`sw t1, 0(a0)`: `t1` → memory (`t1` is the *input*).
In both, the register in parentheses is only used to compute the address.

`sw` writes 4 bytes. `sh` writes the low 2 bytes of the register, and `sb` writes the low byte. Stores don't need signed/unsigned versions: nothing is being extended.

Exercise 4

With `&A[0]` in `a0` (`int A[]`), write RISC-V for `A[2] = A[0];`. (Hint: there is no memory-to-memory move.)

Exercise 5

A student meant to write `*p = x` (`p` in `a0`, `x` in `t1`) but typed `sw a0, 0(t1)`. What does this instruction actually do?

5. Endianness: Which Byte Goes First?

A word is 4 bytes, but memory addresses point to single bytes. So when a word is stored, which of its bytes goes at the lowest address?

- **Little endian:** the *least* significant byte (the "little end") goes first. **RISC-V uses this.**
- **Big endian:** the *most* significant byte (the "big end") goes first.

Example 3: `sw` of `0x12345678` to address `0x100`

Address	0x100	0x101	0x102	0x103
Little endian	78	56	34	12
Big endian	12	34	56	78

Endianness is about the order of **bytes**, never the order of bits inside a byte.

- Endianness only matters when you look at the same memory in *different sizes*, such as storing a word and reading back one byte, or reading a file or network packet written by another machine. If you always store and load whole words, you get the same value back either way.
- RISC-V memory is little-endian by default (the specification also allows big-endian systems, but they are rare).
- Instructions are always stored as little-endian, no matter what the data uses. (The specification stores them in 16-bit little-endian pieces, so a 32-bit instruction also ends up in little-endian byte order.)

Example 4: What `sw t1, 0(a0)` does, one byte at a time

On a little-endian machine, `sw t1, 0(a0)` has the same effect on memory as:

```
sb   t1, 0(a0)      # byte 0 = bits 7-0   (least significant)
srli t2, t1, 8
sb   t2, 1(a0)      # byte 1 = bits 15-8
srli t2, t1, 16
sb   t2, 2(a0)      # byte 2 = bits 23-16
srli t2, t1, 24
sb   t2, 3(a0)      # byte 3 = bits 31-24 (most significant)
```

A big-endian machine would use offsets 3, 2, 1, 0 instead. (Using `t2` for the shifted copies leaves `t1` unchanged, just like the real `sw`.)

Exercise 6

`sw` stores `0xCAFEBABE` at address `0x300`. List the byte at each address `0x300`–`0x303` for a little-endian machine and for a big-endian machine. On RISC-V, what does `lbu t0, 1(a0)` load if `a0` = `0x300`?

6. Register → Register: `mv` and `li`

A **pseudo-instruction** is a convenient name the assembler accepts and quietly replaces with one or more real instructions. It makes code easier to read without adding hardware.

C	Pseudo-instruction	Real instruction(s)
`y = x;` (x in a0, y in t1)	`mv t1, a0`	`addi t1, a0, 0`
`y = 1;`	`li t1, 1`	`addi t1, x0, 1`
`y = 0x12345678;`	`li t1, 0x12345678`	`lui t1, 0x12345` then `addi t1, t1, 0x678`

- `mv` ("move") actually *copies*: the source register keeps its value.
- `li` ("load immediate") puts a constant into a register. It never touches memory, despite the name. Small constants (-2048 to 2047) need one `addi`; larger ones need `lui` + `addi` (details in Lecture 07).

- Others you have met: `ret` = `jalr x0, 0(ra)`; `nop` = `addi x0, x0, 0`.

Exercise 7

Write the real instruction for (a) `mv a0, t0` and (b) `li t0, 100`.

7. Putting It Together

Example 5: `silly_math`

```c
int silly_math(int *a, int b) {
    *a = (b - *a) + 11;
    return *a;
}
```

Step 1: where is everything? `a` (a pointer) is in `a0`, `b` is in `a1`, the return value goes in `a0`. Temporaries `t0`–`t6` are free to use.

Step 2: break the C into one-operation pieces: load `*a`; subtract; add 11; store back; return.

```
silly_math:
    lw   t1, 0(a0)       # t1 = *a
    sub  t0, a1, t1      # t0 = b - *a
    addi t0, t0, 11      # t0 = (b - *a) + 11
    sw   t0, 0(a0)       # *a = t0
    addi a0, t0, 0       # return value = t0   (same as: mv a0, t0)
    ret
```

Step 3: check with numbers. If `*a` = 5 and `b` = 20: `t1` = 5, `t0` = 15, then 26; memory at `a` becomes 26, and the function returns 26.

Watch out

`sw t0, 0(a0)` must come *before* `addi a0, t0, 0`. Once `a0` is overwritten with the result, the pointer `a` is gone and the store would go to the wrong address.

Exercise 8

Translate `void swap(int *a, int *b)` (`a` in `a0`, `b` in `a1`), which exchanges `*a` and `*b`.

Recap

Remember these

1. The processor computes only on registers. Loads bring data in from memory; stores send it back.
2. `lw rd, off(rs1)`: `rd` ← memory[`rs1` + off]. `sw rs2, off(rs1)`: memory[`rs1` + off] ← `rs2`.
3. Displacement addressing: address = base register + 12-bit signed offset.
4. Offsets are in **bytes**: element i of an `int` array is at offset $4i$.
5. `lb`/`lh` sign-extend; `lbu`/`lhu` zero-extend. `sb`/`sh` store the low bits.
6. RISC-V is little-endian: the least significant byte is stored at the lowest address.
7. There is no memory-to-memory move: load into a register, then store.
8. `mv rd, rs` = `addi rd, rs, 0`. `li rd, imm` = `addi` (or `lui` + `addi`). Neither touches memory.
9. To translate C: decide where each variable lives, split the statement into one-operation steps, then check with sample numbers.

Answer key for Exercises

Ex. 1. `lw t2, 0(a1)`.

Ex. 2. `&A[5]` = `0x1000` $+5 \times 4 =$ `0x1014`. `&B[5]` = `0x2000` $+5 \times 2 =$ `0x200A`, loaded with `lh t0, 10(a0)`.

Ex. 3. `lw t0, 12(a0)` `lw t1, 16(a0)` `add t2, t0, t1`. (Check: if `A` $= \{10, 20, 30, 40, 50\}$, `t2` $= 90$.)

Ex. 4. `lw t0, 0(a0)` then `sw t0, 8(a0)`. The value has to pass through a register.

Ex. 5. It stores the value of `a0` (the *pointer* `p`) at the address held in `t1` (the *value* `x`). The roles are swapped: it writes to the wrong place, and most likely crashes because `x` isn't a valid address.

Ex. 6. Little endian: `0x300`: `BE`, `0x301`: `BA`, `0x302`: `FE`, `0x303`: `CA`.
Big endian: `CA`, `FE`, `BA`, `BE`. On RISC-V, `lbu t0, 1(a0)` loads `0x000000BA`.

Ex. 7. (a) `addi a0, t0, 0`. (b) `addi t0, x0, 100`.

Ex. 8.

```
swap:
    lw   t0, 0(a0)       # t0 = *a
    lw   t1, 0(a1)       # t1 = *b
    sw   t1, 0(a0)       # *a = old *b
    sw   t0, 0(a1)       # *b = old *a
    ret
```

Both values must be loaded *before* either store, or one of them is lost.

Student Practice Worksheet

Part 1: Loads, Stores, and Arrays

Q1. `&A[0]` is in `a0` (`int A[]`) and `x` is in `t1`. Write one instruction for each:

- `x = A[0];` _____
- `x = A[7];` _____
- `A[3] = x;` _____

Q2. Memory at address `0x500` (in `a0`) holds the bytes `85 00 7F FF`. Give each result in hex:

- `lb t0, 0(a0)`: _____ `lbu t0, 0(a0)`: _____
- `lh t0, 2(a0)`: _____ `lhu t0, 2(a0)`: _____
- `lw t0, 0(a0)`: _____

Q3. A program accidentally uses `lw t0, 2(a0)` to read `A[2]` of an `int` array. What offset should it use, and why?

Part 2: Endianness

Q4. Memory at `0x400` holds the bytes `01 02 03 04`. What does a 4-byte load return on (a) a little-endian machine: `0x`_____ and (b) a big-endian machine: `0x`_____?

Q5. A `char` variable holds -1 (stored as the byte `0xFF`). What register value do you get with `lb`? With `lbu`? Which one gives back -1?

Part 3: Translate to RISC-V

Q6. `void inc(int *p) { *p = *p + 1; }` (`p` in `a0`)

Q7. `int sum3(int *A) { return A[0] + A[1] + A[2]; }` (`A` in `a0`)

Q8. `void copy_short(short *dst, short *src) { *dst = *src; }` (`dst` in `a0`, `src` in `a1`)

Step-by-Step Solutions & Explanations

Q1: Array access

- `x = A[0];` → `lw t1, 0(a0)`
- `x = A[7];` → `lw t1, 28(a0)` $(7 \times 4 = 28)$
- `A[3] = x;` → `sw t1, 12(a0)` $(3 \times 4 = 12)$

Q2: Loads from `85 00 7F FF`

- `lb 0`: byte `0x85`, top bit 1 → `0xFFFFFF85` (-123). `lbu 0`: `0x00000085` (133).
- `lh 2`: bytes `7F FF` → half-word `0xFF7F` (little endian), top bit 1 → `0xFFFFFF7F` (-129). `lhu 2`: `0x0000FF7F` (65,407).
- `lw 0`: `0xFF7F0085`.

Q3: Wrong offset Answer: Use `lw t0, 8(a0)`. Offsets count bytes, and each `int` is 4 bytes, so `A[2]` starts at byte $2 \times 4 = 8$. Offset 2 would read the second half of `A[0]` and the first half of `A[1]` mixed together (and is a misaligned access).

Q4: Endianness

- (a) Little endian: the byte at the lowest address is the least significant → `0x04030201`.
- (b) Big endian: the byte at the lowest address is the most significant → `0x01020304`.

Q5: Loading a `char` of -1

- `lb`: `0xFFFFFFFF` $= -1$. `lbu`: `0x000000FF` $= 255$.
- `lb` gives back -1, because it sign-extends. (For `signed char` use `lb`; for `unsigned char`, and plain `char` on RISC-V, use `lbu`.)

Q6: `inc`

```
inc:
    lw   t0, 0(a0)       # t0 = *p
    addi t0, t0, 1       # t0 = *p + 1
    sw   t0, 0(a0)       # *p = t0
    ret
```

Q7: `sum3`

```
sum3:
    lw   t0, 0(a0)       # A[0]
    lw   t1, 4(a0)       # A[1]
    add  t0, t0, t1      # A[0] + A[1]
    lw   t1, 8(a0)       # A[2]
    add  a0, t0, t1      # return A[0] + A[1] + A[2]
    ret
```

Check: `A` $= \{3, 4, 10\}$ returns 17.

Q8: `copy_short`

```
copy_short:
    lh   t0, 0(a1)       # t0 = *src   (2 bytes)
    sh   t0, 0(a0)       # *dst = t0   (low 2 bytes)
    ret
```

`lhu` would also work here: `sh` stores only the low 16 bits, so the upper bits don't matter.